

## II. Buy and sell the next day

1. Buy the stock on the fourth day open, if the stock closes down consecutively for three days.
2. Exit on the next day open.
3. Optional: Optimise the strategy by exiting the long position on the same day close. Also, you can optimise the number of down days. There are high chances that the number of down days would be different for each stock.

```
In [2]: import pandas as pd
import numpy as np
import datetime
import warnings
import os
import matplotlib.pyplot as plt
import yfinance as yf
warnings.filterwarnings("ignore")
```

```
In [3]: ticker = "MSFT"
end = datetime.date(2025,12,31)
start = end - pd.Timedelta(days = 365*10)
```

```
In [4]: data1 = yf.download(ticker, start , end , auto_adjust=False)
if isinstance(data1.columns, pd.MultiIndex):
    data1.columns = data1.columns.droplevel('Ticker')
data = data1.copy()
data
```

```
[*****100%*****] 1 of 1 completed
```

Out[4]:

|            | Price      | Adj Close  | Close      | High       | Low        | Open     | Volume |
|------------|------------|------------|------------|------------|------------|----------|--------|
| Date       |            |            |            |            |            |          |        |
| 2016-01-04 | 47.680954  | 54.799999  | 54.799999  | 53.389999  | 54.320000  | 53778000 |        |
| 2016-01-05 | 47.898495  | 55.049999  | 55.389999  | 54.540001  | 54.930000  | 34079700 |        |
| 2016-01-06 | 47.028397  | 54.049999  | 54.400002  | 53.639999  | 54.320000  | 39518900 |        |
| 2016-01-07 | 45.392624  | 52.169998  | 53.490002  | 52.070000  | 52.700001  | 56564900 |        |
| 2016-01-08 | 45.531845  | 52.330002  | 53.279999  | 52.150002  | 52.369999  | 48754000 |        |
| ...        | ...        | ...        | ...        | ...        | ...        | ...      | ...    |
| 2025-12-23 | 483.780823 | 486.850006 | 487.829987 | 484.739990 | 484.980011 | 14683600 |        |
| 2025-12-24 | 484.943420 | 488.019989 | 489.160004 | 484.829987 | 485.679993 | 5855900  |        |
| 2025-12-26 | 484.635406 | 487.709991 | 488.119995 | 485.959991 | 486.709991 | 8842200  |        |
| 2025-12-29 | 484.029236 | 487.100006 | 488.350006 | 484.179993 | 484.859985 | 10893400 |        |
| 2025-12-30 | 484.406860 | 487.480011 | 489.679993 | 485.500000 | 485.929993 | 13944500 |        |

2513 rows × 6 columns

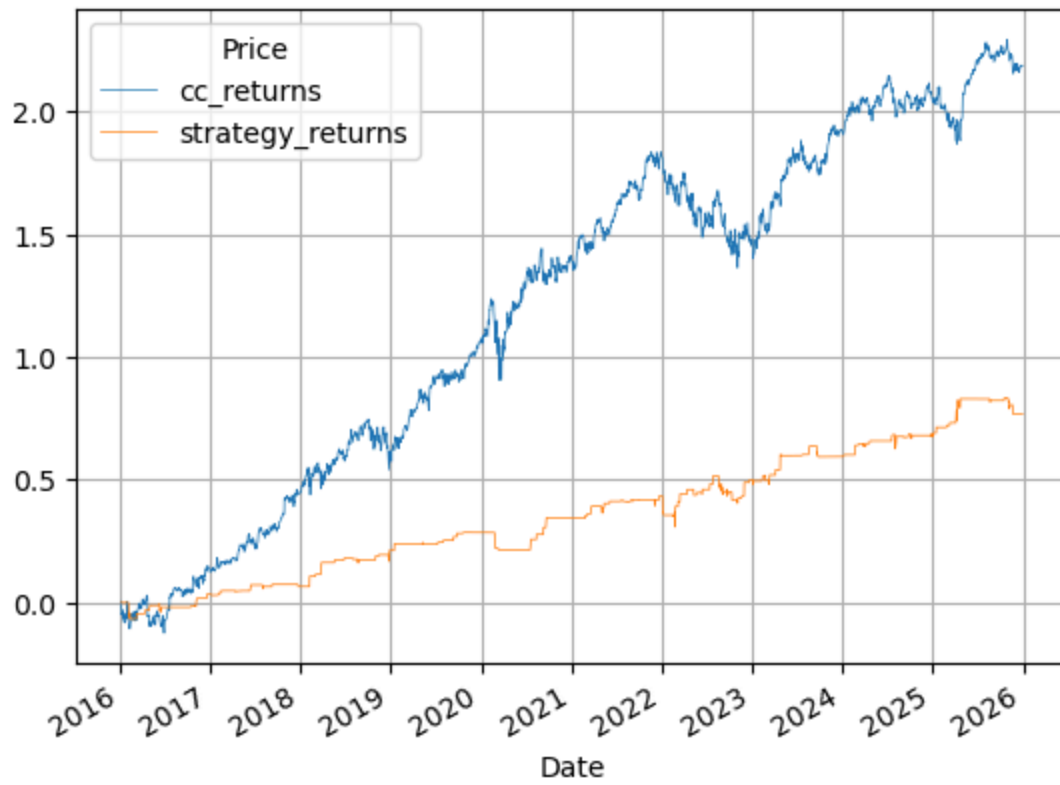
```
In [18]: data["Buy"] = np.where((data["Close"].shift(1) < data["Close"].shift(2))&
                                (data["Close"].shift(2) < data["Close"].shift(3))&
                                (data["Close"].shift(3) < data["Close"].shift(4)), 1, 0)

data["return"] = np.where(data["Buy"] == 1, (data["Open"].shift(-1) / data["Open"] - 1), 0)
```

```
In [22]: data["cc_returns"] = np.log(data["Close"] / data["Close"].shift(1))
data["Position"] = data["Buy"]
data["strategy_returns"] = data["return"]
```

```
In [23]: data[["cc_returns", "strategy_returns"]].cumsum().plot(grid=True, linewidth = 0.5)
```

```
Out[23]: <Axes: xlabel='Date'>
```



In [24]: data

Out[24]:

|            | Price      | Adj Close  | Close      | High       | Low        | Open      | Volume   | Buy | Exit | c |
|------------|------------|------------|------------|------------|------------|-----------|----------|-----|------|---|
| Date       |            |            |            |            |            |           |          |     |      |   |
| 2016-01-04 | 47.680954  | 54.799999  | 54.799999  | 54.799999  | 53.389999  | 54.320000 | 53778000 | 0   | 0.0  |   |
| 2016-01-05 | 47.898495  | 55.049999  | 55.389999  | 54.540001  | 54.930000  | 34079700  | 0        | 0.0 |      |   |
| 2016-01-06 | 47.028397  | 54.049999  | 54.400002  | 53.639999  | 54.320000  | 39518900  | 0        | 0.0 | -    |   |
| 2016-01-07 | 45.392624  | 52.169998  | 53.490002  | 52.070000  | 52.700001  | 56564900  | 0        | 0.0 | -    |   |
| 2016-01-08 | 45.531845  | 52.330002  | 53.279999  | 52.150002  | 52.369999  | 48754000  | 0        | 0.0 |      |   |
| ...        | ...        | ...        | ...        | ...        | ...        | ...       | ...      | ... | ...  |   |
| 2025-12-23 | 483.780823 | 486.850006 | 487.829987 | 484.739990 | 484.980011 | 14683600  | 0        | 0.0 |      |   |
| 2025-12-24 | 484.943420 | 488.019989 | 489.160004 | 484.829987 | 485.679993 | 5855900   | 0        | 0.0 |      |   |
| 2025-12-26 | 484.635406 | 487.709991 | 488.119995 | 485.959991 | 486.709991 | 8842200   | 0        | 0.0 | -    |   |
| 2025-12-29 | 484.029236 | 487.100006 | 488.350006 | 484.179993 | 484.859985 | 10893400  | 0        | 0.0 | -    |   |
| 2025-12-30 | 484.406860 | 487.480011 | 489.679993 | 485.500000 | 485.929993 | 13944500  | 0        | 0.0 |      |   |

2513 rows × 12 columns



In [25]:

data["Position"].plot()

Out[25]: <Axes: xlabel='Date'>



In [ ]: